

CMPU3010: Databases

SQL SUBQUERIES: SIMPLE, CORRELATED, SCALAR

Learning Outcomes

Explain	what a subquery is;
Distinguish	between simple, scalar and correlated subqueries;
Use	subqueries with comparison operators, IN and EXISTS;
Recognise	when a join may be clearer;
Avoid	common subquery errors.

What is a subquery?

A **subquery** is a query nested inside another SQL statement.

```
SELECT ...  
FROM ...  
WHERE column operator (  
    SELECT ...  
    FROM ...  
);
```

The inner query produces a result that is used by the outer query.

```
SELECT service_name, standard_fee  
FROM service  
WHERE standard_fee > (  
    SELECT AVG(standard_fee)  
    FROM service  
);
```

The inner query calculates the average fee.

The outer query returns services costing more than that average.

What is a subquery?

A subquery can be both **simple and scalar**. For example, a query calculating one average value is independent and returns one value.

Type	Description	Typical use
Simple subquery	Runs independently of the outer query	Produce a list or intermediate result
Scalar subquery	Returns exactly one value	Compare a column with an average, maximum or count
Correlated subquery	Refers to a row from the outer query	Test or calculate something separately for each outer row

Simple subquery

A simple or non-correlated subquery can run independently.

Find patients who have at least one completed appointment:

```
SELECT patient_id, first_name,  
last_name  
  
FROM patient  
  
WHERE patient_id IN (  
    SELECT patient_id  
    FROM appointment  
    WHERE status = 'Completed'  
);
```

The inner query runs first:

```
SELECT patient_id  
  
FROM appointment  
  
WHERE status = 'Completed';
```

It produces a list of patient IDs. The outer query then retrieves the corresponding patients.

IN with a subquery

Use IN when the subquery may return multiple values.

Find staff members who provide Physiotherapy:

```
SELECT staff_id, first_name, last_name
FROM staff
WHERE staff_id IN (
    SELECT ss.staff_id
    FROM staff_service AS ss
    WHERE ss.service_id = (
        SELECT service_id
        FROM service
        WHERE service_name =
        'Physiotherapy'
    )
);
```

Important:

- column = (subquery) expects one value.
- column IN (subquery) can accept several values.

Scalar subquery

A scalar subquery must return:

- one column;
- no more than one row.

Find services with a fee above the average:

```
SELECT service_name, standard_fee
FROM service
WHERE standard_fee > (
    SELECT AVG(standard_fee)
    FROM service
);
```

Display every service alongside the overall average:

```
SELECT
    service_name,
    standard_fee,
    (
        SELECT ROUND(AVG(standard_fee), 2)
        FROM service
    ) AS average_fee
FROM service;
```

A scalar subquery can appear in the SELECT, WHERE or HAVING clause.

Scalar subquery - Errors

```
SELECT service_name
FROM service
WHERE service_id = (
    SELECT service_id
    FROM appointment
);
```

The inner query can return several rows, but = expects one value.

PostgreSQL reports:

```
more than one row returned by a
subquery used as an expression
```

How do we resolve this?

- We use IN

Correlated Subquery

A correlated subquery refers to a value from the current row of the outer query.

Show every patient and their number of appointments:

```
SELECT
    p.patient_id,
    p.first_name,
    p.last_name,
    (
        SELECT COUNT(*)
        FROM appointment AS a
        WHERE a.patient_id = p.patient_id
    ) AS appointment_count
FROM patient AS p
ORDER BY p.last_name, p.first_name;
```

The important correlation is:

- WHERE a.patient_id = p.patient_id
- p.patient_id comes from the outer query.
- Conceptually, the database calculates the subquery for each patient.
- PostgreSQL may internally optimise how it performs the query.

Correlated Subquery with EXISTS

Find patients who have never cancelled an appointment:

```
SELECT
    p.patient_id,
    p.first_name,
    p.last_name
FROM patient AS p
WHERE EXISTS (
    SELECT 1
    FROM appointment AS a
    WHERE a.patient_id =
p.patient_id
    AND a.status = 'Cancelled'
);
```

SELECT 1 is conventional because EXISTS only checks whether a matching row exists.

It does not use the returned column value.

Correlated Subquery with NOT EXISTS

Find patients who have never had an appointment:

```
SELECT
    p.patient_id,
    p.first_name,
    p.last_name
FROM patient AS p
WHERE NOT EXISTS (
    SELECT 1
    FROM appointment AS a
    WHERE a.patient_id =
p.patient_id
    AND a.status = 'Cancelled'

);
```

This is often safer than NOT IN, particularly when the subquery could contain NULL.

Correlated Comparison

Find services whose number of appointments is greater than the average number of appointments per service:

```
SELECT
    s.service_id,
    s.service_name
FROM service AS s
WHERE (
    SELECT COUNT(*)
    FROM appointment AS a
    WHERE a.service_id = s.service_id
) > (
    SELECT AVG(service_total)
    FROM (
        SELECT COUNT(*) AS service_total
        FROM appointment
        GROUP BY service_id
    ) AS service_counts
);
```

This example combines:

- a correlated subquery to count appointments for the current service;
- a simple scalar subquery to calculate the average service count;
- a subquery in the FROM clause to create an intermediate result.

Choosing an approach

Requirement	Possible approach
Compare with one calculated value	Scalar subquery
Compare with a list of values	Subquery with IN
Check whether related rows exist	Correlated EXISTS
Check whether related rows do not exist	Correlated NOT EXISTS
Return columns from several tables	Usually a join
Perform calculations for each outer row	Correlated subquery

Summary

A **simple subquery** does not depend on the outer query.

A **scalar subquery** returns one value.

A **correlated subquery** refers to the current outer row.

Use IN for a collection of values.

Use EXISTS or NOT EXISTS to test for related rows.

Always consider whether a join would express the requirement more clearly.